

ASI Platform MVP Technical Architecture Specification

This specification defines the production boundaries, routes, endpoints, database schema, and core logical engines for the launch of the **AI Systems Institute (ASI)** platform, focusing exclusively on **Level 1: AI Systems Strategy**.

1. Directory Route Topology (Pages & Views)

To maintain high SEO discoverability (university scanners checking student credentials) and clean client performance, we outline the exact route layout of the Next.js application.

```

/ (Home / Landing Page)
├── /checkout (Stripe Checkout Redirect)
├── /dashboard (High-Level Progress & Student Roadmap Portal)
├── /workspace (The Parent Route for the Builder Console)
│   ├── /module/1.1 (System Boundaries)
│   ├── /module/1.2 (Threshold Tuning)
│   └── /module/1.3 (Data Ingestion Pipelines)
└── /registry (Public Portfolio Verification Dossier)
    └── /[student-id] (Verifiable Dossier Page)
  
```

A. / (Public Landing & Conversion Portal)

- **Functional Requirements:** Public marketing page highlighting the ASI curriculum, comparative matrix against legacy alternatives, and conversion actions for both the **\$20 Challenge Pass** and the **\$150 Course Bundle**.
- **MVP Implementation:** Static HTML/React. No complex client-side state. Standard Stripe checkout session redirect hooks.

B. /dashboard (The Progress & Roadmap Portal)

- **Functional Requirements:** The central tracking hub for the student. It displays:
 - Overall progress across Level 1 modules.
 - Visual, interactive curriculum roadmap.
 - Academic progress insights (e.g., performance metrics, pending specifications).
 - Direct deep links to resume construction inside `/workspace`.
- **MVP Implementation:** Read-only dashboard components connected directly to the profile state inside Supabase.

- **Functional Requirements:** The dual-panel execution environment. Keeps the student locked in focused execution.
 - **Left Deck (Tabs 1-3):** Renders the static lecture visual metaphor, the interactive system drill widget, and the scaffolded markdown canvas component template.
 - **Right Deck (Tabs 1-2):** Hosts the real-time Socratic AI sidebar chat and the live Markdown workspace editor.
- **MVP Implementation:** The cockpit must actively sync state. The markdown text entered into Left Deck Tab 3 (Mini-Artifact Canvas) must dynamically populate the workspace template in Right Deck Tab 2.

D. /registry/[student-id] (The Verifiable Dossier Registry)

- **Functional Requirements:** Publicly readable page hosting the immutable proof of artifact completion. It must load in under 200 ms to pass university crawling audits.
- **MVP Implementation:** Serves a dynamic rendering of the student's database state. It must display:
 - Student metadata (Name, Class Year, School, Certificate Level status).
 - The verified end-to-end **Technical Specification Document (TSD)** rendered cleanly from the database markdown field.
 - Toggle controls to inspect the three verified constituent Mini-Artifact blocks (Boundary Matrix, Trade-off Guide, Ingestion Flowchart).
 - **Leaderboard Deferral:** The leaderboard block is completely omitted. Instead, this page is framed strictly as an academic credential registry verification page.

2. API Endpoint Directory

All endpoints must be authenticated using the client JWT token passed via Supabase session headers, except for the public registry reading endpoint and the Stripe webhook.

A. Authentication & Gateways

POST /api/checkout/create-session

- **Payload:**

```
{
  "gateType": "challenge" | "bundle",
  "moduleLevel": 1
}
```

- **Returns:** { "url": "https://checkout.stripe.com/..." }
- **Action:** Generates a Stripe Checkout Session.

- **Payload:** Raw Stripe webhook event signature.
- **Action:** On `checkout.session.completed`, parses the metadata and updates the Supabase `profiles.tier` state to 'bundle' or increments `profiles.challenge_tokens`.

B. Socratic AI Engine

POST `/api/socratic/chat`

- **Payload:**

```
{
  "moduleId": "1.2",
  "message": "User statement text...",
  "activeWorkspaceMarkdown": "Current state of student's workspace brief..."
}
```

- **Returns:** Streamed text response (Server-Sent Events) from Claude 3.5 Haiku.
- **Action:** Appends the student's active workspace state into the system prompt context, ensuring the Socratic coach can critique the student's actual work without revealing templates.

C. The Grading & Compile Engine

POST `/api/grade/evaluate`

- **Payload:**

```
{
  "moduleId": "1.2",
  "miniArtifactType": "tradeoff_guide",
  "markdownContent": "Raw markdown text written by student..."
}
```

- **Returns:**

```
{
  "passed": true | false,
  "feedback": "Strict Socratic validation critique text explaining structural"
}
```

- **Action:** Triggers the LLM structural audit. If passed, writes the compiled block into the public `mini_artifacts` registry storage block.

3. Database Schema (Supabase PostgreSQL)

We keep our schema highly flat and robust to avoid query performance overhead.

+-----+		+-----+		+-----+
profiles		mini_artifacts		major_artifacts
+-----+		+-----+		+-----+
id (UUID, PK)	<-----+	id (UUID, PK)		id (UUID, PK)
full_name		profile_id (FK)	<-----+	profile_id (FK)
school		module_id		level_id
class_year		artifact_type		compiled_markdown
tier		content_markdown		status
challenge_tokens		status		verified_at
+-----+		+-----+		+-----+
		verified_at		
		+-----+		

PostgreSQL Table DDL Specifications

-- 1. Profiles Table

```
CREATE TABLE public.profiles (
  id UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,
  full_name TEXT NOT NULL,
  school TEXT NOT NULL,
  class_year INTEGER NOT NULL,
  tier TEXT DEFAULT 'free' CHECK (tier IN ('free', 'challenge', 'bundle')),
  challenge_tokens INTEGER DEFAULT 0 CHECK (challenge_tokens >= 0),
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- 2. Mini-Artifacts Table

```
CREATE TABLE public.mini_artifacts (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  profile_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
  module_id TEXT NOT NULL, -- e.g., '1.1', '1.2'
  artifact_type TEXT NOT NULL, -- e.g., 'boundary_matrix', 'tradeoff_guide'
  content_markdown TEXT NOT NULL,
  status TEXT DEFAULT 'draft' CHECK (status IN ('draft', 'verified')),
  verified_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (profile_id, module_id, artifact_type)
);
```

-- 3. Major Artifacts Table

```
CREATE TABLE public.major_artifacts (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  profile_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
  level_id INTEGER NOT NULL CHECK (level_id = 1),
  compiled_markdown TEXT NOT NULL,
  status TEXT DEFAULT 'draft' CHECK (status IN ('draft', 'verified')),
  verified_at TIMESTAMPTZ,
  UNIQUE (profile_id, level_id)
);
```

4. Operational Requirements Matrix: MVP vs. Phase 2

To maintain an uncompromised launch schedule, we divide technical execution boundaries cleanly:

System Layer	MUST WORK (MVP Release)	DEFER / PLACEHOLDER (Phase 2+)
Authentication	Native Supabase OAuth/Email Login & RLS rules protecting users' workspaces.	Third-party school roster single sign-on (SSO) integrations.
Workspace Editor	Auto-saving Markdown text editor that buffers content to LocalStorage before writing back to Supabase.	Collaborative, real-time cursor editing or multi-student sharing.
Socratic Chat	Continuous API calls routing user query context plus workspace state through Claude 3.5 Haiku.	Vector memory databases (RAG) tracking students' overall historical tone.
Programmatic Lockout	None. Immediate free retry attempts allowed.	Database lockout enforcing a 48 hours delay after 3 consecutive failed grading passes.
The Public Registry	High-performance, SEO-friendly page loading static cached portfolios of students displaying verified artifacts.	Dynamic system optimization telemetry charts showing latencies, costs, and accuracies.
Interactive Drills	Client-side visual widgets (sliders, demographic click toggles) to solidify concepts before building.	Persistent database storage tracing every single slider tick or path interaction.
Leaderboards	None. The entire leaderboard paradigm is removed from the MVP.	Sorting algorithms matching metrics weight algorithms:

Score = w_a · Acc – w_c · Cost – w_l · Lat

5. Automated Socratic Auditing Engine Prompt

When POST /api/grade/evaluate is called, the system passes this instruction to Claude 3.5 Haiku. The model parses the raw user markdown to evaluate completeness and structural correctness without human intervention.

SYSTEM PROMPT: Automated Structural Auditing Engine

TASK: Audit student Technical Specification Brief (TSB) markdown contents for log

EVALUATION RULES:

1. Ensure all mandatory headers exist (e.g., "# Machine Boundary Definitions", "#
2. Validate that inputs and output vectors are defined as variables, not written
3. Verify that the student has committed to a specific optimization choice (Recal and has written at least two sentences logically justifying their choice based
4. If the markdown is copy-pasted template syntax or contains empty placeholders the evaluation MUST fail.

OUTPUT FORMAT:

The model must output a JSON object containing the exact properties mapped in the

```
{
  "passed": true | false,
  "feedback": "Socratic explanation of what structural gaps exist in the student"
}
```

6. Immediate Next Steps / Future Enhancements

Once the core MVP platform is stable, the following features should be integrated into the codebase immediately to protect model costs and drive retention mechanics.

A. Programmatic Lockout Integration (Rate Limiting)

To prevent students from brute-forcing the grading engine with trial-and-error scripts:

1. Create the `lockout_states` table in PostgreSQL:

```
CREATE TABLE public.lockout_states (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  profile_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
  module_id TEXT NOT NULL,
  failed_attempts INTEGER DEFAULT 0 NOT NULL,
  locked_until TIMESTAMPTZ,
  UNIQUE (profile_id, module_id)
);
```

2. Update `POST /api/grade/evaluate` to check if `locked_until > NOW()`. If so, abort with a **423 Locked** response.
3. If an evaluation fails, increment `failed_attempts`. Upon reaching **3** consecutive failed attempts on a single module, set `locked_until = NOW() + INTERVAL '48 hours'`.

B. System Optimization Leaderboard Integration

Layer in the competitive global leaderboard by writing student telemetry optimizations into a permanent `leaderboard_scores` table, sorted dynamically via:

$$\text{Score} = w_a \cdot \text{Accuracy} - w_c \cdot \text{OperationalCost} - w_l \cdot \text{Latency}$$

Where w_a , w_c , and w_l are weights representing the business priority of the chosen modular system context.